# ShadowMem — Forensic & Detection Engine

## System Error Diagnosis, Technical Root Causes & AI Remediation Guide

**Target Codebase:** c:\Users\Himesh\Desktop\ShadowMem | **Remediation Agent:** FreeBuff AI / Pair Engineer | **Audit Date:** 2026-09-15 11:33:26

This engineering report outlines all diagnosed bugs, runtime exceptions, test suite logic errors, and architectural inconsistencies identified in the ShadowMem project. Each item includes the exact file path, line numbers, failure reproduction trace, underlying root cause, and copy-paste ready rectified code.

## 1. Executive Error Summary Matrix

| ID | Severity | Target File | Issue Description | Impact & Status |
|---|---|---|---|---|
| **ERR-01** | **High** | detecter.py:118, 131<br>make_guide.py:246 | UnicodeEncodeError on Windows cp1252 printing checkmark character (\u2713 / ✓). | Console scripts crash instantly when clean scan finishes or guide is written. |
| **ERR-02** | **High** | test_sandbox.py:185, 249-263 | Variable shadowing 'for name, passed in results' overwriting pass counter + NoneType.get(). | Reports '5 TESTS FAILED' even when all pass; raises AttributeError on clean scans. |
| **ERR-03** | **High** | app.py:1929 | Frontend containment UI adds 'pointer-events-none' to modalIsolateBtn when suspended. | Locks out operator from clicking 'Resume Process' once a PID has been isolated. |
| **ERR-04** | **Medium** | test_fixes.py:6<br>malware_simulator.py:366 | Hardcoded timeout=30 in urllib.request.urlopen for /api/scan endpoint. | Full memory sweep of 130+ processes takes ~33.5s; causes artificial TimeoutError. |
| **ERR-05** | **Medium** | malware_simulator.py:107, 341 | REFRESH_INTERVAL=5.0 spamming 33s scans + calling /simulate/alert on app.py. | CPU pegged at 100% due to overlapping scans; returns HTTP 404 for missing route. |
| **ERR-06** | **Medium** | server.py vs app.py index.html & trigger_test_event.py | Two disconnected server architectures: mock WebSocket server vs real Win32 forensics. | Standalone index.html and trigger_test_event.py fail when launch_shadowmem.bat runs app.py. |
| **ERR-07** | **Low** | test_injector.py:298 Project Root | Missing CloseHandle(spin_thread) + corrupted filenames with Windows colons (C■Users...). | Handle leak during test execution and filesystem pollution from unquoted bash redirection. |

## 2. Deep Technical Breakdown & Remediation Instructions

## ERR-01: Windows Console UnicodeEncodeError Crash (detecter.py & make_guide.py)

**Affected Files:** detecter.py (lines 118, 131), make_guide.py (line 246)

**Root Cause:** On Microsoft Windows, the standard console stdout defaults to code page 1252 (cp1252). The Unicode character \u2713 (the visual checkmark '✓') cannot be mapped to any cp1252 byte value. When run_detection() in detecter.py completes without detecting malware, it attempts to print '\n[✓] Scan complete...', crashing with UnicodeEncodeError: 'charmap' codec can't encode character '\u2713'. Similarly, make_guide.py crashes on line 246 upon writing presentation_guide.html.

**Reproduction Exception Trace:**

### Exception Output

```
Traceback (most recent call last):
  File "C:\Users\Himesh\Desktop\ShadowMem\detecter.py", line 136, in <module>
    run_detection()
  File "C:\Users\Himesh\Desktop\ShadowMem\detecter.py", line 131, in run_detection
    print("\n[\u2713] Scan complete: No active fileless injections detected.")
UnicodeEncodeError: 'charmap' codec can't encode character '\u2713' in position 3: character maps to <undefined>
```

**Rectification Plan (FreeBuff AI Instructions):**

- In detecter.py: change line 118 to verdict = "[OK] CLEAN" and line 131 to print("\n[+] Scan complete: No active fileless injections detected.").
- In make_guide.py: change line 246 to print("[+] Updated presentation_guide.html with complete master guide!").

### Rectified Code (detecter.py & make_guide.py)

```
# In detecter.py line 118:
        else:
            verdict = "[OK] CLEAN"
            top_technique = None

# In detecter.py line 130-131:
    if threat_count == 0:
        print("\n[+] Scan complete: No active fileless injections detected.")

# In make_guide.py line 246:
print("[+] Updated presentation_guide.html with complete master guide!")
```

## ERR-02: Variable Shadowing, AttributeError & Thread Inflation (test_sandbox.py)

**Affected File:** test_sandbox.py (lines 185-200, 249-263)

**Root Cause 1 (Shadowing Bug):** In main(), line 249 calculates passed = sum(1 for _, p in results if p). Immediately afterward at line 252, the loop for test_name, passed in results: reuses the identifier passed. In Python, this loop variable overwrites the integer counter with the boolean result of the final test (True). At line 256, it prints 'Total: True/6 test suites passed'. At line 258, if passed == total: evaluates True == 6 (False). At line 262, f'*** {total - passed} TESTS FAILED' evaluates 6 - True = 5, falsely announcing that 5 tests failed and exiting with error code 1!

**Root Cause 2 (NoneType AttributeError):** In Test 6 (test_technique_classification), classification = result.get('classification', {}) is called. Since 'classification': None exists in the dict, dict.get() returns None (not {}). Calling classification.get('display_name') crashes with AttributeError: 'NoneType' object has no attribute 'get'.

**Root Cause 3 (Thread Count Explosion):** When running all tests in sequence, scikit-learn, numpy, and psutil accumulate worker threads in the runner's own PID (reaching 45 threads). The ML model was trained with beacons at ~9 threads and benign multi-threaded apps at ~35 threads. With 45 threads, the model suppresses the beacon score down to 8.29%, yielding status='Clean' and classification=None.

### Rectified Code (test_sandbox.py)

```
# 1. Add at top of test_sandbox.py (before importing numpy/sklearn/app):
import os
os.environ['OMP_NUM_THREADS'] = '1'
os.environ['OPENBLAS_NUM_THREADS'] = '1'
os.environ['MKL_NUM_THREADS'] = '1'

# 2. In test_technique_classification() line 185:
    classification = result.get('classification') or {}

# 3. In main() lines 249-263 (fix shadowing):
    passed_count = sum(1 for _, p in results if p)
    total_count = len(results)
    for test_name, test_status in results:
        status = 'PASS' if test_status else 'FAIL'
        print(f'  [{status}] {test_name}')
    print(f'\nTotal: {passed_count}/{total_count} test suites passed')
    if passed_count == total_count:
        print('\n*** ALL TESTS PASSED - Sandbox is working correctly ***')
        return 0
    else:
        print(f'\n*** {total_count - passed_count} TESTS FAILED - Review needed ***')
        return 1
```

## ERR-03: Process Containment Button UI Lockout (app.py)

**Affected File:** app.py (line 1929)

**Root Cause:** When an operator opens a process dossier in the web dashboard and isolates it, setIsolateUI(pid, true) executes. Line 1929 adds the Tailwind CSS utility pointer-events-none to modalIsolateBtn while setting its label to 'Resume Process'. The pointer-events-none CSS rule completely disables mouse click events on that element. As a result, the operator can never click the button again to resume the suspended process; the UI remains locked out until page refresh.

**Original Buggy Snippet:**

**Buggy Code in app.py**

```
function setIsolateUI(pid, suspended) {
  if (suspended) {
    isolatedPids.add(pid);
    modalIsolatedBadge.classList.remove('hidden');
    modalIsolateBtn.classList.add('opacity-50', 'pointer-events-none'); // <--- CAUSES PERMANENT LOCKOUT
    modalIsolateBtn.innerHTML = '...<span>Resume Process</span>';
```

**Rectification Plan (FreeBuff AI Instructions):**

- Remove 'pointer-events-none' from line 1929. The button should remain clickable so the operator can invoke toggleIsolate with currentlySuspended=true.

**Rectified Code (app.py)**

```
// In app.py line 1926-1935 (setIsolateUI):
function setIsolateUI(pid, suspended) {
  if (suspended) {
    isolatedPids.add(pid);
    modalIsolatedBadge.classList.remove('hidden');
    modalIsolateBtn.classList.remove('pointer-events-none'); // Ensure clickable
    modalIsolateBtn.innerHTML =
      '<svg class="w-3.5 h-3.5 text-ok" fill="none" viewBox="0 0 24 24" stroke="currentColor" stroke-width="2">'
      + '<path stroke-linecap="round" stroke-linejoin="round" d="M5 13l4 4L19 7" />'
      + '</svg>'
      + '<span>Resume Process</span>';
  } else { ... }
```

## ERR-04: Socket Timeout Failures on Full Memory Scan (test_fixes.py)

**Affected Files:** test_fixes.py (line 6), malware_simulator.py (line 366)

**Root Cause:** /api/scan iterates through all user processes via Win32 VirtualQueryEx and NtQueryInformationThread. On a live Windows 11 system with 130-150 processes, this forensic walk takes ~33.5 seconds. In test_fixes.py, line 6 sets timeout=30. The HTTP client aborts with a TimeoutError: timed out even though the API is working perfectly. (Note: test_scan_api.py was previously patched to 45s for this exact reason).

**Rectification Plan (FreeBuff AI Instructions):**

- In test_fixes.py line 6, change timeout=30 to timeout=60.
- In malware_simulator.py line 366, change timeout=30 to timeout=60.

**Rectified Code (test_fixes.py & malware_simulator.py)**

```
# In test_fixes.py line 6:
r = urllib.request.urlopen('http://127.0.0.1:8000/api/scan', timeout=60)

# In malware_simulator.py line 366:
with urllib.request.urlopen(req, timeout=60) as resp:
```

### ERR-05: Simulation Scan Loop Flooding & Endpoint Mismatch (malware_simulator.py)

**Affected File:** malware_simulator.py (lines 104, 107, 341-357)

**Root Cause:** By default, DISPLAY_MODE = 'both' and REFRESH_INTERVAL = 5.0. Every 5 seconds, the background thread executes trigger_api_scan(), which calls /api/scan. Because a full memory scan takes 33 seconds, background threads rapidly accumulate, triggering multiple concurrent full-system forensic sweeps. This pins the CPU to 100% and starves the Uvicorn event loop. Additionally, post_websocket_alert() targets /simulate/alert, which returns HTTP 404 when running app.py.

**Rectification Plan (FreeBuff AI Instructions):**

- Change REFRESH_INTERVAL = 45.0 so scans do not overlap.
- Set DISPLAY_MODE = 'scanner' when running against the primary app.py forensic engine.

**Rectified Code (malware_simulator.py)**

```
# In malware_simulator.py lines 104-107:
# Options: 'alerts' (server.py WebSocket), 'scanner' (app.py REST API), or 'both'
DISPLAY_MODE = 'scanner'  # Use 'scanner' for app.py prototype

# Set interval greater than full scan duration (~33s) to prevent CPU thread starvation
REFRESH_INTERVAL = 45.0
```

### ERR-06: Dual Divergent Backend Architectures (server.py vs app.py)

**Affected Files:** server.py, app.py, index.html, trigger_test_event.py, launch_shadowmem.bat

**Root Cause:** The project contains two divergent implementations: 1. **Real Engine (app.py):** Runs the genuine Win32 memory scanner (memoryscanner.py), the trained ML pipeline (shadowmem_model.pkl), provides containment endpoints (/api/isolate, /api/resume, /api/audit), and serves an embedded modern Tailwind UI at GET /. 2. **Mock Server (server.py):** Streams synthetic JSON alerts over /ws/alerts for the standalone index.html file, and exposes /simulate/alert. The launch batch script (launch_shadowmem.bat) starts app.py. Therefore, opening the standalone index.html fails to connect to WebSockets, and running trigger_test_event.py fails with HTTP 404 Not Found.

**Rectification Recommendation:**

- Treat app.py as the definitive application. Either migrate server.py's /simulate/alert and /ws/alerts into app.py, or update documentation/scripts to clarify that server.py is a mock demo.

### ERR-07: Handle Leak in test_injector.py & Corrupted Shell Artifacts

**Affected Files:** test_injector.py (line 298), Root Directory

**Root Cause 1 (Handle Leak):** In test_injector.py, a raw spin thread is launched at line 242. In the finally teardown block (line 298), kernel32.TerminateThread(spin_thread, 0) is called, but kernel32.CloseHandle(spin_thread) is omitted, leaving an unclosed kernel thread handle.

**Root Cause 2 (Junk Files):** The root directory contains filenames resulting from unquoted bash redirection on Windows: C■UsersHimeshDesktopShadowMemcurl_home.html, ...uvicorn_inline.log, ...uvicorn_start.log, and .uvicorn8080.pid. These should be purged.

**Rectified Code (test_injector.py)**

```
# In test_injector.py line 298-302:
        try:
            if kernel32.TerminateThread(spin_thread, 0):
                print("[*] Raw spin thread terminated.")
            kernel32.CloseHandle(spin_thread) # Added handle cleanup
        except Exception:
            pass
```

## 3. Remediation Verification Checklist

After FreeBuff AI applies the code rectifications above, run the following verification commands in order:

- [ ] **Verify Console Encoding:** Run python detecter.py. It must complete without UnicodeEncodeError.
- [ ] **Verify Sandbox Test Suite:** Run python test_sandbox.py. All 6 test suites must PASS and exit with code 0.
- [ ] **Verify Live API Scan:** Run python test_fixes.py (while app:app is running). It must complete all tests without timeouts.
- [ ] **Verify Containment UI:** In browser, open http://127.0.0.1:8000, scan, inspect an anomaly, click *Isolate Process*, and verify the button changes to *Resume Process* and remains clickable.